

Projeto 2: Sistema de Assinatura Digital e Verificação Segura de Arquivos com RSA-OAEP e RSA-PSS sobre SHA3-256

Roberto Ribeiro Correa De Oliveira Neto — 242009936

Daniel de Oliveira Morais — 242042396

Davi Bragança e Silva — 242001473

Departamento de Ciência da Computação, Universidade de Brasília (UnB), Brasília, DF

Profa. Priscila Solís Barreto

Abstract/Resumo—Este trabalho apresenta uma implementação sem OpenSSL de RSA-2048 com Miller-Rabin, cifragem RSA-OAEP e assinatura RSA-PSS, ambas com SHA3-256 e MGF1, e um formato de assinatura destacada de arquivos. Testes de adulteração confirmam a detecção de alterações no arquivo, na assinatura e na chave.

Keywords—RSA, OAEP, PSS, SHA3-256

I. INTRODUÇÃO

A segurança em redes de computadores depende de três propriedades quando os dados atravessam meios não confiáveis: confidencialidade, integridade e autenticidade. A criptografia de chave pública permite cifrar mensagens sem segredo compartilhado prévio e assinar documentos de modo que qualquer pessoa possa verificá-los, sendo a base de protocolos como TLS, de certificados digitais e da distribuição segura de software [1].

Este projeto, desenvolvido na disciplina, implementa do zero, em Python e sem OpenSSL, um sistema de assinatura digital e verificação de arquivos. São cobertos: a geração de chaves RSA de no mínimo 2048 bits com o teste de Miller-Rabin; a cifragem RSAES-OAEP; a assinatura RSASSA-PSS; e uma estrutura de assinatura destacada em Base64 com verificação de integridade. O hash é o SHA3-256 [2] e a máscara é a MGF1, ambos parametrizados conforme a RFC 8017 [1].

O restante do relatório está organizado assim: a Seção II resume os fundamentos teóricos; a Seção III descreve o cenário experimental e analisa os resultados de cada atividade; a Seção IV apresenta as conclusões, seguida da bibliografia.

II. FUNDAMENTAÇÃO TEÓRICA

A. RSA e geração de chaves

No RSA, o módulo é $n = pq$, com p e q primos, e o expoente privado é $d = e^{-1} \bmod \lambda(n)$, com $\lambda(n) = \text{mmc}(p-1, q-1)$. Cifra-se com $c = m^e \bmod n$ e decifra-se com $m = c^d \bmod n$; a assinatura usa a mesma operação com os papéis das chaves trocados [1]. O RSA "de livro-texto" é determinístico e maleável, e por isso nunca deve ser aplicado diretamente às mensagens: é preciso um esquema de codificação como OAEP ou PSS.

Os primos são gerados por sorteio de candidatos testados com Miller-Rabin: escreve-se $n-1 = 2^s d$ com d ímpar, e uma base aleatória a é testemunha de composição se $a^d \bmod n$ for diferente de 1 e de $n-1$ e nenhum dos quadrados sucessivos for igual a $n-1$. Cada rodada erra com probabilidade de no máximo $1/4$ para um composto qualquer, de modo que 40 rodadas garantem erro de no máximo 2^{-80} . As normas [3], [4] exigem ainda $\text{mdc}(p-1, e)$

$$= 1, |p-q| > 2^{n_{\text{len}}/2-100}, d > 2^{n_{\text{len}}/2} \text{ e } 2^{16} < e < 2^{256}.$$

B. CRT e ataques de implementação

A operação privada é acelerada pelo Teorema Chinês dos Restos: calculam-se $m_1 = x^{dP} \bmod p$ e $m_2 = x^{dQ} \bmod q$, e recombina-se com $h = q \text{Inv}(m_1 - m_2) \bmod p$ e $m = m_2 + hq$. Implementações ingênuas são vulneráveis a ataques de temporização, mitigados com *blinding* (multiplicar a entrada por r^e aleatório e remover r ao final) [5], e a ataques por falha no CRT, mitigados verificando o resultado com a chave pública antes de devolvê-lo [6].

C. SHA3-256 e MGF1

O SHA3-256 [2] produz digests de $hLen = 32$ bytes. A MGF1 expande uma semente em uma máscara de tamanho arbitrário: $T = \text{Hash}(semente \parallel C(0)) \parallel \text{Hash}(semente \parallel C(1)) \parallel \dots$, em que $C(i) = \text{I2OSP}(i, 4)$ [1].

D. RSAES-OAEP

Com k o tamanho do módulo em bytes, o bloco é $DB = \text{lHash} \parallel PS \parallel 0x01 \parallel M$, seguido de $\text{maskedDB} = DB \text{ xor } \text{MGF1}(seed)$, $\text{maskedSeed} = seed \text{ xor } \text{MGF1}(\text{maskedDB})$ e $EM = 0x00 \parallel \text{maskedSeed} \parallel \text{maskedDB}$. A semente aleatória torna a cifragem probabilística, e o rótulo (*label*) é autenticado via lHash . A mensagem tem no máximo $k-2hLen-2$ bytes (190 bytes para RSA-2048). Como diferenças entre falhas de decodificação permitem o ataque de Manger [7], toda falha deve produzir um único erro, sem distinguir a causa.

E. RSASSA-PSS

No PSS [1], [8] o digest $mHash$ passa por uma codificação probabilística antes da operação privada: $M' = 0x00^8 \parallel mHash \parallel salt$, $H = \text{Hash}(M')$, $DB = PS \parallel 0x01 \parallel salt$, $\text{maskedDB} = DB \text{ xor } \text{MGF1}(H)$ e $EM = \text{maskedDB} \parallel H \parallel 0xBC$, com $emBits = modBits-1$. Portanto, assinar **não** é "cifrar o hash com a chave privada": o salt faz duas assinaturas do mesmo documento serem diferentes, e a verificação refaz H a partir do digest e do salt recuperado.

III. AMBIENTE EXPERIMENTAL E ANÁLISE DE RESULTADOS

A. Descrição do Cenário

Hardware e software. Os experimentos foram executados em Linux x86_64, processador Intel Xeon @

2,10 GHz (1 vCPU), 3,9 GB de RAM, com Python 3.12.3. Não há dependências externas: usam-se apenas *hashlib* (SHA3-256), *secrets* (CSPRNG), *hmac* (comparação em tempo constante), *base64* e *json*. O *mdc*, o Euclides estendido, o inverso modular e a exponenciação modular (quadrados sucessivos) são implementações próprias, e o *pow* do Python foi usado apenas para conferir os resultados.

Topologia. O cenário é o de duas partes em um canal inseguro: o signatário (Alice) gera o par de chaves, assina o arquivo e envia arquivo e .sig; o verificador (Bob) só conhece a chave pública e recalcula o digest do arquivo recebido. Um adversário no canal pode alterar o arquivo, a assinatura ou tentar substituir a chave pública. Não houve rede física, pois a análise trata dos mecanismos criptográficos e dos formatos, e os arquivos transitam localmente. A Tabela I mostra os módulos.

TABELA I
MÓDULOS DO SISTEMA

Módulo	Função
aritmética	mdc, Euclides estendido, inverso e exp. modular, I2OSP/OS2IP
primos, chaves	Miller-Rabin, geração, validação, importação/exportação
primitivas	RSAEP/RSADP/RSASP1/RSAP1 com CRT, <i>blinding</i> e verificação
hashing	SHA3-256 (arquivos em blocos) e MGF1
oaep, pss	Codificação e decodificação OAEP e PSS
assinatura_arquivo	Estrutura .sig, <i>parsing</i> estrito e verificação
cli	Comandos: gerar-chaves, info, cifrar, decifrar, assinar, verificar

Formatos. As chaves são gravadas em envelope textual (BEGIN/END RSASEG ... KEY) com JSON canônico em Base64 e inteiros em hexadecimal; a chave privada é criada com permissão 0600. O arquivo .sig traz cabeçalho (algoritmo, hash, MGF, tamanho do salt, *fingerprint* da chave, nome, tamanho e digest do arquivo) e a assinatura em Base64. Os metadados são apenas informativos: a validade depende exclusivamente da verificação PSS sobre o digest **recalculado** do arquivo.

B. Análise de Resultados

Atividade 1 — Geração de chaves (Parte I). Em 20 gerações de RSA-2048, o tempo médio foi de 1,13 s (desvio-padrão 0,46 s; mínimo 0,48 s; máximo 2,03 s). A variação é esperada, pois a densidade de primos exige um número aleatório de candidatas. Duas escolhas reduzem o custo e evitam repetições: um crivo por primos menores que 2000 antes do Miller-Rabin e a fixação dos dois bits mais altos de p e q , o que garante n com exatamente 2048 bits. As chaves geradas têm p e q de 1024 bits, $e = 65537$ e d com 2046 bits (acima de 2^{1024}), e passam por *validar()*, que confere $n = pq$, a primalidade, $ed = 1 \bmod \lambda(n)$ e os parâmetros do CRT. O Miller-Rabin rejeitou os números de Carmichael 561 e 1729 e aceitou o primo de Mersenne $2^{127}-1$, e a exponenciação e o inverso próprios coincidiram com o *pow* do Python em entradas de 2000 bits.

Atividade 2 — Cifragem OAEP (Parte II). A mensagem "segredo" foi cifrada e decifrada corretamente, gerando texto cifrado de 256 bytes. Uma mensagem de

191 bytes foi recusada (*MensagemMuitoLonga*), confirmando o limite de 190 bytes. Duas cifragens da mesma mensagem produziram textos diferentes, como esperado do OAEP. Ao inverter um bit do texto cifrado, ou ao decifrar com rótulo diferente, obteve-se em ambos os casos o mesmo erro genérico "falha na decifragem", sem revelar a causa e portanto sem criar oráculo de padding.

Atividade 3 — Assinatura e adulteração (Partes III e IV). Um arquivo de teste de 2000 bytes foi assinado e verificado. Depois se aplicaram as três adulterações pedidas, além de uma estrutura malformada. A Tabela II resume os resultados, todos idênticos aos esperados. O diagnóstico em (a) e (c) usa metadados não autenticados e é só informativo, mas o veredito de invalidez vem sempre da verificação PSS.

TABELA II
TESTES DE ADULTERAÇÃO

Cenário	Resultado obtido
Arquivo e assinatura originais	ASSINATURA VÁLIDA
(a) 1 byte do arquivo alterado	INVÁLIDA: arquivo modificado após a assinatura
(b) 1 bit da assinatura invertido	INVÁLIDA: assinatura corrompida ou forjada
(c) chave pública de outro par	INVÁLIDA: chave não corresponde à do signatário
Arquivo .sig malformado	INVÁLIDA: estrutura malformada

Atividade 4 — Desempenho. A Tabela III traz as médias de 100 a 200 execuções por operação (20 no caso da geração de chaves), com o tamanho de mensagem e de digest típicos. A verificação e a cifragem são cerca de 55 vezes mais rápidas que as operações privadas porque $e = 65537$ exige poucas multiplicações. Em relação à exponenciação direta com d (33,8 ms), o CRT com *blinding* e verificação (11,8 ms) é cerca de 2,9 vezes mais rápido, ou seja, as duas proteções foram pagas pelo ganho do CRT. A exponenciação própria também ficou cerca de 1,5 vez mais lenta que o *pow* nativo (22,0 ms), custo aceitável por ser parte do objetivo didático. Para um arquivo de 50 MB, assinar levou 0,18 s (uma execução), dominado pelo cálculo do digest em blocos de 64 KiB, de modo que o custo RSA é praticamente constante em relação ao tamanho do arquivo.

TABELA III
TEMPOS DAS OPERAÇÕES (RSA-2048)

Operação	Tempo médio $\pm$ dp (ms)
Cifrar (OAEP)	0,21 $\pm$ 0,04
Decifrar (OAEP)	11,56 $\pm$ 0,22
Assinar digest (PSS)	11,63 $\pm$ 0,99
Verificar (PSS)	0,21 $\pm$ 0,04
Op. privada sem CRT (direta)	33,79 $\pm$ 2,84
Op. privada CRT + <i>blinding</i> + verif.	11,77 $\pm$ 1,73

Testes automatizados. O projeto inclui uma suíte de testes unitários (*unittest*) que cobre aritmética, primalidade, chaves, OAEP, PSS, adulteração e CLI, além de um teste opcional de interoperabilidade com bibliotecas consolidadas, usado apenas para conferir a saída.

Limitações e ajustes. Python não garante tempo constante; por isso a temporização é mitigada por *blinding* e as decodificações OAEP e PSS evitam desvios

dependentes de dados secretos e usam `hmac.compare_digest`. A chave privada é armazenada sem senha, protegida apenas pela permissão 0600. O *fingerprint* identifica a chave, mas não vincula a chave a uma identidade, o que exigiria certificados. Por fim, o verificador aceita somente os parâmetros fixos implementados (por exemplo, salt de 32 bytes), sem negociar valores vindos do arquivo, o que elimina ataques de troca de algoritmo.

IV. CONCLUSÕES

Foi implementado, sem OpenSSL, um sistema completo de RSA-2048 com OAEP e PSS sobre SHA3-256, cobrindo geração de chaves, cifragem, assinatura e verificação de arquivos. Os experimentos mostraram que qualquer alteração no arquivo, na assinatura ou na chave é detectada e classificada, e que o OAEP responde a todas as falhas com um único erro. O CRT reduziu em cerca de 2,9 vezes o custo da operação privada mesmo com *blinding*. Como limitações, restam a ausência de tempo constante em Python e a chave privada sem senha; como trabalho futuro, propõem-se proteção por senha e certificados.

BIBLIOGRAFIA

- [1] K. Moriarty, B. Kaliski, J. Jonsson e A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," RFC 8017, IETF, nov. 2016.
- [2] NIST, "SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions," FIPS PUB 202, ago. 2015.
- [3] NIST, "Digital Signature Standard (DSS)," FIPS PUB 186-5, fev. 2023.
- [4] E. Barker *et al.*, "Recommendation for Pair-Wise Key-Establishment Using Integer Factorization Cryptography," NIST SP 800-56B Rev. 2, mar. 2019.
- [5] P. Kocher, "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems," em *CRYPTO 1996*, LNCS 1109, pp. 104-113.
- [6] D. Boneh, R. DeMillo e R. Lipton, "On the importance of checking cryptographic protocols for faults," em *EUROCRYPT 1997*, LNCS 1233, pp. 37-51.
- [7] J. Manger, "A chosen ciphertext attack on RSA OAEP as standardized in PKCS #1 v2.0," em *CRYPTO 2001*, LNCS 2139, pp. 230-238.
- [8] M. Bellare e P. Rogaway, "The exact security of digital signatures: how to sign with RSA and Rabin," em *EUROCRYPT 1996*, LNCS 1070, pp. 399-416.